

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“Jnana Sangama”, Belagavi – 560018.



A MINI PROJECT REPORT

ON

“GESTURE CONTROLLED VIRTUAL MOUSE”

Submitted in partial fulfillment for the award of
Bachelor of Engineering in Computer Science and Engineering

By

CHANDU S R

1CK23CS030

FAIZUNNISA

1CK23CS039

HARSHA S K

1CK23CS049

HESHA P

1CK23CS054

Under the Guidance of

Prof. SWATHI J K

Asst. Professor, Dept. of CS&E



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

C. BYREGOWDA INSTITUTE OF TECHNOLOGY

Approved by AICTE New Delhi, Accredited by NAAC with B++ Grade, Recognized by
Govt. of Karnataka, Affiliated to VTU Belagavi, ISO 9001:2015 Certified Institute
Kolar – Srinivaspur Road,

Kolar – 563101

2025-2026

C BYREGOWDA INSTITUTE OF TECHNOLOGY

**Approved by AICTE New Delhi, Accredited by NAAC with B++ Grade, Recognized
by Govt. of Karnataka, Affiliated to VTU Belagavi, ISO 9001:2015 Certified Institute
Kolar-Srinivasapura road,
Kolar-563101**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the Mini Project work entitled **“GESTURE CONTROLLED VIRTUAL MOUSE”** is a bonafide work carried out by **CHANDU SR (USN:1CK23CS030), FAIZUNNISA (USN:1CK23CS039), HARSHA SK (USN:1CK23CS049), HESHA P (USN:1CK23CS054)** in partial fulfillment for the award of Bachelor of Engineering in Computer Science & Engineering of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of Mini Project work prescribed for the V Semester Bachelor of Engineering Degree.

Signature of the Guide
Prof. Swathi J K
Asst. Professor,
Dept. of CSE

Signature of the HOD
Dr. Vasudeva R
Assoc. Prof. & HOD,
Dept. of CSE

Signature of the Principal
Dr. S N Chandrashekara
Principal,
CBIT-Kolar

ABSTRACT

In this project, we develop a gesture-controlled virtual mouse system that enables users to interact with their computer through hand gestures instead of conventional input devices like a mouse or keyboard. The system is implemented using Python, OpenCV, and MediaPipe, which together enable accurate real-time hand detection and tracking using a standard webcam. Each recognized gesture is mapped to specific mouse operations such as cursor movement, left click, right click, double click, and screenshot capture, allowing seamless and intuitive control of the computer.

This approach eliminates the need for physical contact, offering a more hygienic and convenient alternative for device interaction, particularly useful in environments where touch-based interfaces are impractical. The system's design is focused on improving accessibility, flexibility, and efficiency, making it beneficial for users with physical disabilities as well as for general hands-free applications.

Furthermore, this project demonstrates the potential of computer vision and artificial intelligence in enhancing human-computer interaction (HCI). By leveraging advanced gesture recognition algorithms, the system ensures reliable and responsive operation even in dynamic lighting and background conditions. Overall, the proposed gesture-controlled virtual mouse represents a step toward creating smart, contactless, and user-friendly control systems for future computing environments.

DECLARATION

We, **CHANDU SR** bearing the **USN 1CK23CS030**, **FAIZUNNISA** bearing **USN 1CK23CS039**, **HARSHA SK** bearing **USN 1CK23CS049**, **HESHA P** bearing **USN 1CK23CS054**, students of 5th semester B.E, Computer Science and Engineering of VTU, declare that this project entitled **"GESTURE CONTROLLED VIRTUAL MOUSE"**, embodies the report of the project work carried out under the guidance of **SWATHI JK, Asst Professor Dept of CSE,CBIT** as the partial fulfilment of the requirement of the award of the degree in Bachelor of engineering, Computer Science and engineering, affiliated to Visvesvaraya Technological university, Belagavi during academic year 2025-2026. Further the content embodies in the project has not been submitted previously by anybody for the award of any other degree.

Place: Kolar

Date:

SIGNATURE OF STUDENT

CHANDU SR

FAIZUNNISA

HARSHA SK

HESHA P

ACKNOWLEDGEMENT

The completion of project brings with and sense of satisfaction, but it is never completed without thanking the persons who are all responsible for its successful completion.

First and foremost. We wish to express our deep sincere feelings of gratitude to my Institution, **C BYREGOWDA INSTITUTE OF TECHNOLOGY**, for providing mean opportunity to do our education.

We extend our deep sense of sincere gratitude to our beloved principal **Dr. S N CHANDRASHEKARA**, for permitting to carry out the project work successfully.

We extend our heartfelt sincere gratitude to **Dr. VASUDEVA R**, Assoc. Professor and HOD, Department of Computer Science Engineering, CBIT for giving us guidance, valuable advice and support.

We extend our sincere gratitude to our Mini project coordinator **Mrs. SARIKA C G**, Assistant Professor, Department of computer science and engineering, for all their prolific support, guidance and valuable suggestions at all stages of this complete process.

We extend our sincere gratitude to our internal guide **Mrs. SWATHI J K**, Assistant Professor, Department of computer science and engineering, CBIT for her valuable advice, support and constructive suggestions.

We would like to thank all the Teaching and non-teaching staff members of Department of Computer Science and Engineering, for their co-operation and suggestions.

Finally, we like to thank our parents and friends for their continuous encouragement and moral support.

CHANDU S R (1CK23CS030)

FAIZUNNISA (1CK23CS039)

HARSHA SK (1CK23CS049)

HESHA P (1CK23CS054)

CONTENTS

Abstract	I
Declaration	II
Acknowledgement	III
Table of Contents	IV
List of Figures	VI
List of Tables	VII

Chapter No	Chapter Name	Page No
1	Introduction	1
	1.1 Introduction to project	1
	1.2 Objectives	1
	1.3 Problem Statement	2
	1.4 Methodology	2
	1.4.1 Steps involved in methodology	4
	1.5 Organisation of Report	5
2	Literature Survey.	6
	2.1 Introduction	6
	2.2 Related Work.	7
3	System Requirement Specification	9
	3.1 Introduction	9
	3.2 Hardware Requirements	10
	3.3 Software Requirements	10
	3.4 Functional Requirements	10
	3.5 Non Functional Requirements	10
	3.6 Preliminary Investigation	11
	3.7 System Environment	12
4	System Analysis	13
	4.1 Existing System	13
	4.2 Proposed System	14
5	System Design	15
	5.1 Input Design	15
	5.2 Output Design	16

5.3	System Architecture	17
5.4	Data Flow Diagrams	19
5.5	Use Case Diagrams	20
6	Implementation	21
6.1	Introduction	21
6.2	Language Selection	22
6.3	Platform Selection	24
6.4	Implemented Modules	25
6.5	Pseudocode	27
7	Testing	31
7.1	Types Of Tests	31
7.2	Unit Testing	31
7.3	Integration Testing	33
7.4	System Testing	34
7.5	Acceptance Testing	34
7.6	Test Cases	34
8	Conclusions and Future Enhancements	37
8.1	Conclusions	37
8.2	Future Enhancements	37
	Bibliography	38
	Appendix A-Acronyms	39
	Appendix B- SNAPSHOTS	40

LIST OF FIGURES

Figure no	Figure Name	Page No
1.1	Hand Detection Tracking	3
1.2	Methodology Flowchart	4
4.1	Proposed System Flowchart	14
5.1	Input design	16
5.2	Output design	17
5.3	System architecture	18
5.4	Data-flow diagram	19
5.5	Use-case diagram	20
7.1	Use-case diagram	32

LIST OF TABLES

Table No	Table name	Page No
7.1	Hand detection and cursor control	35
7.2	Gesture Sensitivity and Accuracy	36

CHAPTER 1

INTRODUCTION

1.1 Introduction

Artificial Intelligence (AI) enables machines to imitate human intelligence by learning, analyzing, and making decisions similar to humans. Virtual Reality (VR), on the other hand, creates immersive, computer-generated 3D environments that users can interact with in a realistic way. When combined, AI and advanced computer-vision techniques open new possibilities in Human-Computer Interaction by making digital systems more natural, intelligent, and accessible.

This project focuses on creating a gesture-controlled virtual mouse that allows users to move the cursor on the screen without using any physical hardware, such as a traditional mouse. Instead, the system tracks hand and finger movements in real time using a camera. By applying techniques such as object detection, image processing, and colour/feature recognition, the system interprets gestures and converts them into mouse actions.

The project uses OpenCV, a widely-used Python library for computer vision, to perform tasks such as background subtraction, fingertip detection, and convex-hull-based gesture recognition. Through these techniques, the system identifies hand shapes and motions to enable cursor movement, clicking, and other interactions.

Traditional mice can be limiting or difficult for users with mobility challenges. A gesture-controlled virtual mouse offers a more intuitive, touch-free, and accessible method for interacting with computers, making it a promising alternative in modern Human-Computer Interaction.

1.2 Objectives

- The primary objective of this project is to design and develop a reliable, efficient, and user-friendly gesture-controlled virtual mouse system that can accurately detect and interpret hand and finger movements to control mouse actions in real time, providing a convenient and accessible alternative to traditional mouse devices.
- The goal of this project is to move the mouse pointer on the screen without using any hardware, such as a mouse, and instead by utilising finger motions, a technique known as gesture recognition. Different technologies have been explored in the development of virtual mice in recent years.
- Our project's suggested technology focuses on three main areas: object identification, picture processing, and colour recognition. We will demonstrate a unique way to Human Computer Interaction in which cursor movement is controlled by a real-time camera in this project.

1.3 Problem Statement

The proposed AI virtual mouse using hand gesture technology can also be introduced in places where there is not enough space to use a physical mouse or for individuals who have difficulties with hand grip and are unable to operate a real mouse. Moreover, in situations like the COVID outbreak, it is not safe to use shared devices by physically touching them, as this can contribute to the spread of infections.

The AI virtual mouse helps overcome all these problems since it uses hand-gesture recognition and fingertip detection to perform mouse actions using only a camera or built-in webcam. Unlike wired, wireless, or Bluetooth mice that depend on external hardware and may stop working due to technical faults, the gesture-controlled virtual mouse provides a more reliable, contactless, and convenient method to interact with the computer.

1.4 Methodology

Pre-processing or to be specific picture handling is an earlier advance in PC vision, where the objective is to change over a picture into a structure reasonable for additional investigation. Instances of tasks, for example, openness rectification, shading adjusting, picture sound decrease, or expanding picture sharpness are exceptionally significant and very consideration requesting to accomplish adequate outcomes. For this article, I propose to introduce a part of the typically used picture taking care of methodology using an outstandingly notable Computer Vision library, Open-CV. I'll endeavor to portray immediately the manner by which each movement works and spotlight more on dealing with the point even more basically, giving you all the code you truly need so you have a functioning experience of the material. with the point even more basically, giving you all the code you truly need so you have a functioning experience of the material.

1.4.1 The steps in methodology are: -

The gesture-controlled virtual mouse system uses a transformational mapping technique to convert the fingertip coordinates detected on the camera frame into corresponding positions on the full-screen computer display, allowing accurate mouse control. A specific rectangular interaction zone is created on the camera feed that represents the computer window, and within this area the user can move their fingertip to guide the mouse pointer across the screen. Once the hand is detected, the system analyzes which fingers are raised or performing a gesture, and based on that, it identifies which mouse operation—such as clicking, dragging, or scrolling—should be executed.

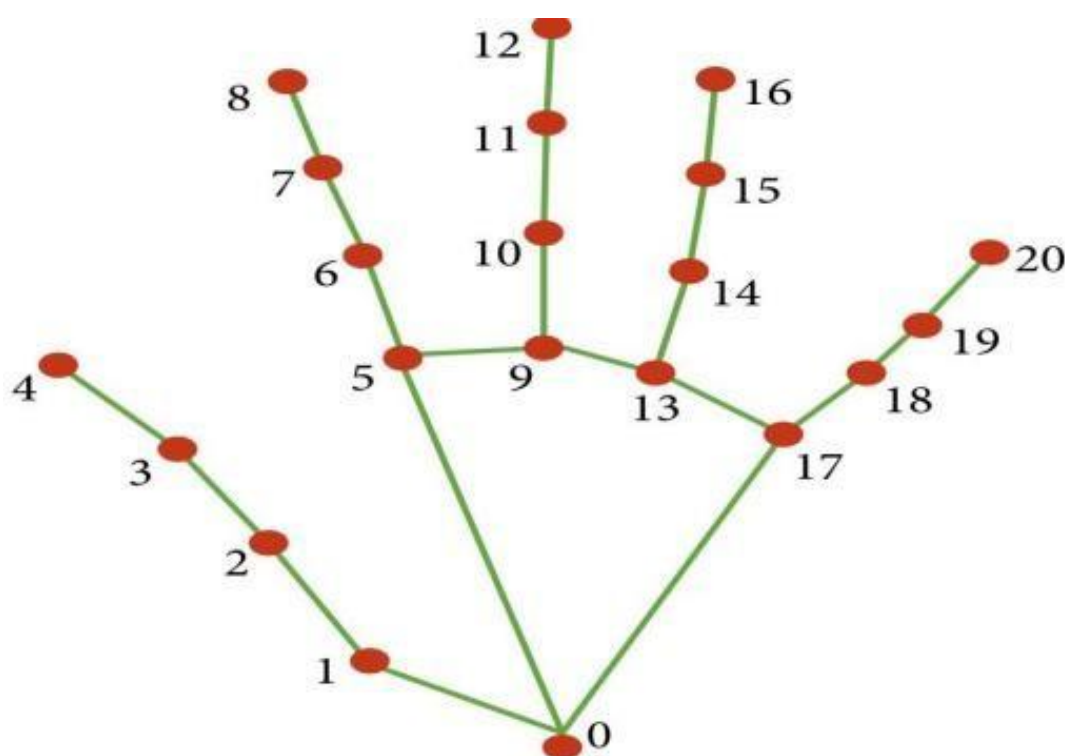


Figure 1.1 Hand Detection Tracking

As shown in the above figure 1.1, the graph is connected, undirected and contains 21 vertices and 20 edges. The edges are drawn in green. The nodes are numbered from 0 to 20 and are drawn as red circles.

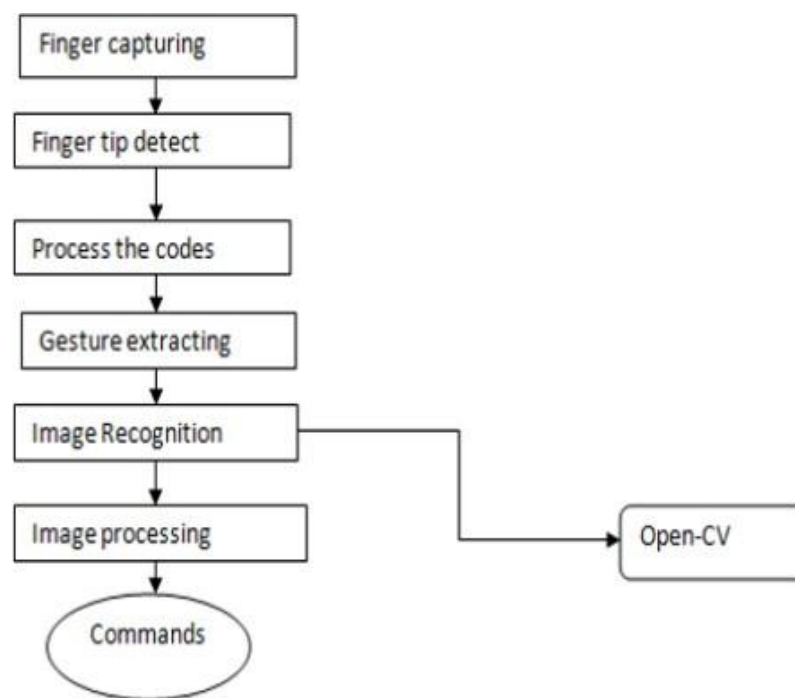


Figure 1.2 Methodology Flowchart

The above Figure 1.2 depicts a flowchart illustrating the general process of recognising hand gesture using Open-CV.

1.5 Organization of Report

Chapter 1: It includes the basic introduction of the project problem statement includes the issues that our project addresses including the main aim of the project. The Methodology have methods that have been used to achieve its goal.

Chapter 2: In this chapter, the fundamental knowledge behind Virtual Gesture Hand Control is discussed. This chapter presents an in-depth review of various research studies and papers, focusing on different classifiers, techniques, and methods used by researchers. These approaches help us understand the existing technologies and provide guidance on selecting suitable techniques that can be applied in our project.

Chapter 3: In this chapter, the main purpose was to explain step by step process and System Requirement specifications which includes hardware, software and other functional requirements.

Chapter 4: In this chapter, the report outlines the desired future state of the Gesture-Controlled Virtual Mouse system based on the identified problems and potential improvements. It also describes the current system in detail, including its components, workflow, data processing method

Chapter 5: In this chapter, the report provides a detailed and structured approach to designing the Gesture-Controlled Virtual Mouse system in a way that meets user requirements and ensures efficiency, accuracy, and easy maintenance by: Translating the functional and technical requirements into a clear and well-defined system design.

Chapter 6: In this chapter, it describes the implementation process which acts as a crucial phase as it provides transparency, reproducibility, and accountability, allowing readers to verify and build upon the results.

Chapter 7: In this chapter, it describes the different format of testing phases, which ensures that the final product meets the required quality, functionality, and performance standards, detects and fixes defects, and ultimately saves time and resources by identifying and addressing issues early on, thereby reducing the risk of delivering a faulty or incomplete product to end-users.

Chapter 8: In this chapter, it includes conclusion and future enhancement in a system development project, organizations can ensure that their system remains effective, efficient, and relevant over time, and that they are well-positioned to take advantage of new opportunities and challenges.

CHAPTER 2

LITERATURE SURVEY

The integration of hand-gesture recognition and virtual interaction systems has the potential to transform various domains, including human–computer interaction, accessibility solutions, gaming, and remote operations. This chapter presents an overview of the fundamental concepts, techniques, and applications of gesture-based control systems, along with the major challenges and limitations currently faced in this field.

2.1 Introduction

The convergence of Artificial Intelligence (AI) and Virtual Reality (VR) is poised to revolutionize multiple industries and redefine how we interact with information. AI, capable of learning, reasoning, and human-like interaction, enhances VR experiences, while VR offers immersive environments that simulate real-world scenarios. Together, AI and VR are driving a new generation of applications across education, healthcare, entertainment, and manufacturing.

The latest breakthroughs in AI and VR are transforming virtual environments, making them more interactive and realistic than ever. With AI, it is now possible to create detailed 3D models, simulate

complex behaviors, and offer personalized experiences. At the same time, VR technologies—such as head-mounted displays and haptic feedback devices—have become more affordable and widely accessible, allowing a larger audience to explore immersive, lifelike virtual worlds.

Although interest in AI and VR continues to grow, a comprehensive literature survey summarizing the current state of research in these fields is still needed. This survey seeks to address that gap by offering a thorough review of existing studies on AI and VR. It will cover key concepts, methodologies, and applications, as well as highlight the current challenges and limitations within these domains.

2.2 Related work

Dung-Hua Liou and Chen-Chiung Hsieh.[1] proposed A Real Time Hand Gesture Recognition System Uses Motion History Image In this study, a face based skin colour model and a motion history image based hand movement detection technique were developed. Hands travelling up, down, left, right are the dynamic hand gestures as well as two static hand gestures the fist and the wave hand are suggested in this work. These hand gestures are simple and effortless. Using Harr-like features, the four-directional dynamic hand movements were identified. Static hand movements were extracted using the face-based adaptive skin colour model, and these motions were subsequently recognised by examining a face-based ROI. To test our suggested system, five people were invited. Experimental results revealed an average accuracy of 94.1% and proved the viability of the suggested approach.

S.Shriram. [2] The current construction is contained a nonexclusive mouse and track pad screen control framework, as well as the mishap of a hand development control structure. The utilization of a hand development to get to the screen from a nice ways is unimaginable. No matter what how it is basically attempting to execute, the degree is just restricted in the virtual mouse field. The current virtual mouse control structure contains direct mouse tasks utilizing a hand attestation framework, in which we have some control over the mouse pointer, left click, right snap, and drag etc. The utilization of hand confirmation in the future won't be utilized. Despite how there are a gathering of frameworks for hand certification, the construction they utilized is static hand attestation, which is just a confirmation of the shape made by the hand and the meaning of activity for each shape made, which is restricted to a few depicted activities and makes a great deal of unsettling influence. As progression drives, there are something else and more decisions rather than utilizing a mouse. Coming up next are a piece of the techniques that were used:-

1) Camera Used in the Virtual Gesture Mouse project: Open- CV is python vision library that contains Associate in the organized AI virtual mouse structure depends upon the edges that are gotten by the camera in Associate in nursing passing computer.

2) Providing Input: Pictures in Computer Vision are portrayed as associations of numbers watching out for the discrete eclipsing or power values present in each picture pixel. Each picture is considered as information displayable in various ways, whether as collections of pixel values or either complex plots keeping an eye on the course of pixel powers.

3) Moving hand through the Window using rectangular area: The AI virtual mouse structure uses the informative algorithmic rule, and it changes over the co-ordinates of tip from the camera screen to the pc window full screen for the mouse.

3) Detect the Finger tips and doing the Mouse Cursor improvements

4) In this construction, AI mouse is police evaluation that finger is up deceiving the spot co-ordinate of the particular finger that it'll found abuse the Media-Pipe and along these lines the specific bits of the fingers that region unit up, and according to that, the authentic mouse perform is played out its assignments.

CHAPTER 3

SYSTEM REQUIREMENTS SPECIFICATION

A System Requirement Specification is a document that defines the technical requirements necessary for a system to function effectively. It serves as a crucial reference in software development projects, ensuring that all stakeholders have a clear and shared understanding of the system's needs, functionalities, and constraints.

3.1 Introduction

A System Requirements Specification (SRS) is a formal document that defines the expected features, behavior, and functionality of a system or software application. It serves as a blueprint for both developers and stakeholders, ensuring a shared understanding of the system's objectives and constraints.

An SRS outlines the main business processes the system will support, the assumptions made during design, and the key performance and quality parameters that must be met. Depending on the development methodology, the level of detail may vary, but it generally includes functional requirements, non-functional requirements, software specifications, and hardware specifications.

3.1.1 Specification definition

This specification documents the system-level requirements for the GPM system.

Specification objectives

The objectives of this specification of the GPM are to:

- **Define** the system's goals and requirements.
- **Measure** the system's performance using quantifiable metrics.
- **Achieve** the system's premises and constraints.
- **Ensure** consistency and clarity in the system's design.
- **Provide** a common understanding of the system's architecture.
- **Guide** system development and implementation.

3.2 Software Requirements

- Operating system : Windows XP / 7/8/10
- Coding Language : Python
- IDE : Visual Studio Code, Python Interpreter.

3.3 Hardware Requirements

- System : Intel i3 processor
- Hard Disk : Minimum 1 GB
- Ram : Minimum 6 GB
- A computer or device with a camera (e.g., webcam)

Hardware requirements are the most common set of requirements defined by any operating system or software application is the physical computer resources, also known as hardware, A hardware requirements list is often accompanied by a hardware compatibility list (HCL), especially in case of operating systems.

3.4 Functional Requirements

Functional Requirement defines a function of a software system and how the system must behave when presented with specific inputs or conditions. These may include calculations, data manipulation and processing and other specific functionality.

In this system following are the functional requirements: -

- User Interface, the system shall provide a user-friendly interface that tracks the user's hand movements in 3D space using VR sensors.
- Gesture Recognition, the system shall maintain a library of predefined gestures and their corresponding volume control actions.
- VR Integration, the system shall integrate with VR environments to provide an immersive experience
- Performance Requirements, the system shall process gestures in real-time. with a latency of less than 100ms.

3.5 Non-Functional Requirements

Non-Functional Requirements (NFRs) are requirements that define the criteria for evaluating the operation of a system, rather than specifying particular functions or behaviors. They focus on the overall qualities and performance of the system, including aspects such as reliability, response time, scalability, maintainability, and storage capacity. NFRs help ensure that the system not only performs its intended functions but also meets standards for efficiency, usability, and robustness. Non-functional requirements arise through the user needs, because of budget constraints, organizational policies, the need for interoperability with other software and hardware systems or because of external factors such as: -

- Product Requirements
- Performance Requirements
- Security Requirements
- Reliability Requirements
- Maintainability Requirements

Non-functional requirements describe how a system must behave and establish constraints of its functionality. This type of requirements is also known as the system's quality attributes. Attributes such as performance, security, usability, compatibility are not the feature of the system, they are a required characteristic. They are "developing" properties that emerge from the whole arrangement and hence we can't compose a particular line of code to execute them. Any attributes required by the customer are described by the specification. We must include only those requirements that are appropriate for our project.

3.6 Preliminary Investigation

Preliminary Investigation is an initial study carried out to understand the feasibility and requirements of a project. This project aims to develop a gesture-controlled virtual mouse, replacing traditional input devices like a physical mouse or touchpad. Conventional devices can be limiting, especially in scenarios requiring hands-free or more intuitive interaction. By using hand gestures, users can control cursor movement, perform left and right clicks, scroll, and interact with applications naturally. This approach enhances accessibility, makes computer interaction more inclusive, and provides a modern, innovative way of controlling computers without relying on physical devices.

3.7 System Environment

The system environment for the gesture-controlled virtual mouse using AI and computer vision comprises both hardware and software components that work together to enable intuitive, hands-free interaction. Hand movements are captured using a camera or depth-sensing device, such as a webcam or Leap Motion controller, and sent to a high-performance computer equipped with a dedicated processor, sufficient RAM, and a graphics card capable of processing real-time gesture recognition. The AI-powered gesture recognition algorithm analyzes the input to detect specific hand gestures and maps them to corresponding mouse actions, such as cursor movement, clicks, and scrolling. The virtual mouse control software translates these recognized gestures into system-level commands, while the operating system provides the necessary interface for controlling cursor and other functions. This integrated setup ensures low-latency, accurate, and smooth interaction, eliminating the need for traditional physical input devices and enabling a more intuitive computing experience.

CHAPTER 4

SYSTEM ANALYSIS

System analysis is "the process of studying a procedure in order to identify its goals and purposes and create [systems](#) and procedures that will achieve them in an efficient way". Another view sees system analysis as a [problem-solving](#) technique that breaks down a system into its component pieces for the purpose of the studying how well those component parts work and interact to accomplish their purpose.

System analysis is used in every field where something is developed. Analysis can also be a series of components that perform organic functions together, such as system engineering. [System engineering](#) is an [interdisciplinary field](#) of engineering that focuses on how complex engineering projects should be designed and managed.

4.1 Existing System

Traditional computer interaction largely depends on physical input devices like a mouse or touchpad. While these devices perform their functions well, they require direct manual operation, which can be cumbersome or restrictive in certain situations, such as when hands-free operation is needed. Some existing gesture-based systems, such as the Microsoft Kinect, demonstrate how hand movements can be tracked using depth-sensing cameras. However, these solutions are expensive

and rely on specialized hardware, making them less accessible to the general user. Recently, research in computer vision and machine learning has advanced the development of hand gesture recognition systems, providing a more natural and intuitive way for users to interact with computers. Various studies on gesture-controlled virtual mice have explored methods such as motion and color detection, depth-sensing cameras, and AI-driven algorithms, showing the potential for creating accurate, responsive, and user-friendly alternatives to conventional input devices.

4.2 Proposed System

An open-source and commonly used library for real-time hand tracking and gesture recognition is MediaPipe, which accurately detects hand landmarks and finger positions. The OpenCV (cv2) library is used for capturing camera frames, processing images, and detecting hand movements effectively. PyAutoGUI enables the system to simulate mouse actions such as cursor movement, clicking, scrolling, and dragging based on the gestures recognized. The random library can be used for generating random values when needed, such as testing gesture responses or adding optional random behaviors for debugging or experimentation. These libraries work together to build a smooth and responsive gesture-controlled system. MediaPipe offers highly accurate tracking, while cv2 processes live video efficiently. PyAutoGUI converts gestures into real mouse actions on the screen. The random library helps in testing dynamic conditions or adding variations during development for better system reliability.

Advantages Of Proposed System:

- i. User-Friendly
- ii. Convenient
- iii. More Engaging



Figure 4.1: Proposed System Flowchart

The above Figure 4.1 depicts a flowchart of a system that uses a camera as input. The camera input is fed into a "Detect Module", which processes the input. The output of the "Detect Module" is then fed into a "Mouse Function", which performs some operation based on the detected information. Finally, the output of the "Mouse Function" is displayed as "Output". This system could be used for various applications, such as controlling a mouse cursor with hand gestures, or creating a virtual keyboard on a screen.

CHAPTER 5

SYSTEM DESIGN

A systemic approach is essential for designing a hand gesture-controlled virtual mouse, ensuring all components work coherently. Both Bottom-Up and Top-Down approaches can be applied to address variables like hand tracking, gesture recognition, and cursor control. Designers use modeling languages to structure the system consistently, representing it either graphically through flowcharts

or UML diagrams, or textually with algorithms and pseudocode. This structured design ensures gestures are accurately translated into virtual mouse actions, providing an intuitive and efficient user interaction.

5.1 Input Design

The system uses a webcam to capture the user's hand and OpenCV to analyze hand shape, orientation, and movement for gesture recognition. Users can perform gestures to control the virtual mouse, such as moving the cursor, performing left/right clicks, or scrolling. Hands are detected using a pre-trained model, and the hand shape is analyzed with convex hull analysis to identify finger positions. Hand orientation is determined using Hough transform, and hand movement is tracked with optical flow analysis. The recognized gestures are then translated into corresponding virtual mouse actions in real time.

The system uses OpenCV's Cascade Classifier to detect hands in the video feed. This pre-trained model employs Haar-like features to identify hands in various orientations and lighting conditions. Hand movement is tracked using optical flow analysis, which calculates the motion of pixels between consecutive frames. By analyzing the optical flow, the system determines the hand's movement and translates it into corresponding actions for controlling the virtual mouse.

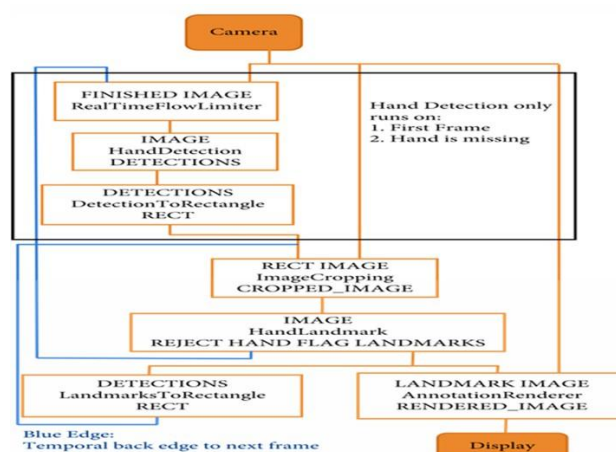


Figure 5.1: Input design

This figure 5.1 appears to show the processing steps for a hand detection and tracking system. The system starts with a camera as the input source. The camera's output is processed to create a 'finished image', then further processed to detect hands. If the hands are detected, they are processed further, to be cropped and then processed for landmark detection. If no hands are found, this processing path does not run. Finally, the output of the landmark detection stage is used to create a rendered image. The rendered image is likely outputted to a display. This appears to be a simplified representation of the system, highlighting the key stages of the process.

5.2 Output design

The design of output is the most important task of any system. During output design, developers identify the type of outputs needed, and consider the necessary output controls and prototype report layouts. The system prioritizes hand detection only in specific scenarios (first frame or hand missing). This suggests a focus on efficiency, avoiding unnecessary computations. The pipeline involves cropping the image to the hand region, emphasizing on analyzing only the relevant area. The final output image includes landmark annotations, indicating a goal of detailed hand analysis.

CAMERA: Capture a frame from the camera.

FINISHED IMAGE: Apply RealTimeFlowLimiter to the captured image.

IMAGE: Detect hands in the image using HandDetection.

DETECTIONS: Convert hand detections to rectangles.

RECT IMAGE: Crop the image to only contain the detected hand.

IMAGE: Detect landmarks within the cropped hand region.

REJECT HAND FLAG LANDMARKS: Check if the hand is valid (not a false positive) by checking the hand landmarks.

DETECTIONS: Convert landmarks to rectangles.

LANDMARK IMAGE: Render the landmarks onto the image.

DISPLAY: Display the final image with rendered landmarks.

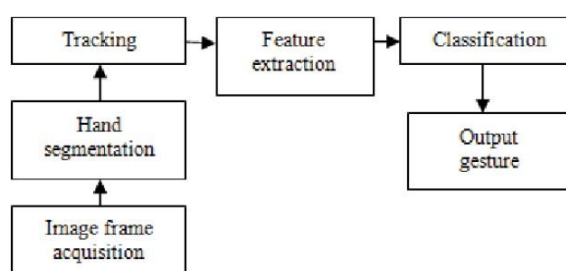


Figure 5.2: Output design

The process starts with a camera taking an image. Then, the image is processed through a real-time flow limiter. Next, a hand detection algorithm is applied. If the hand is detected, the image is cropped to the hand area. Otherwise, the process stops. Then, hand landmarks are detected, and if there are no hand landmarks, the process ends. Finally, the image is rendered with the detected landmarks and displayed. The blue edge indicates the temporal back edge to the next frame as shown in the figure 5.2.

5.3 System Architecture

System architecture refers to the high-level structure of a system, including its components, relationships, and interactions. It defines how the system is organized, how it behaves, and how it interacts with its environment. System architecture is a critical aspect of system design, as it provides a blueprint for building, integrating, and maintaining complex systems.

Communication: It provides a common understanding of the system among stakeholders, including developers, users, and maintainers.

Design guidance: It provides a framework for designing and building the system, ensuring consistency and coherence.

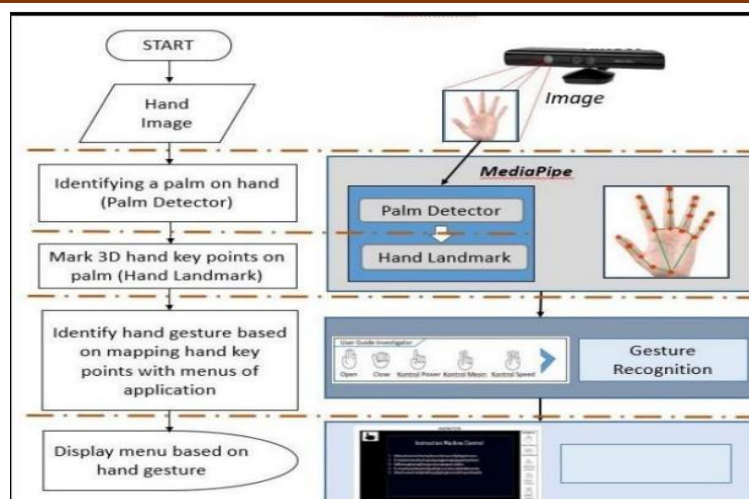


Figure 5.3: System architecture

The figure 5.3 flowchart illustrates a system for recognizing hand gestures and using them to control menus in an application. The process starts with capturing an image of a hand, then identifies the palm and key points on the hand. The hand gesture is then identified based on the key points, and a menu is displayed based on the detected gesture. The image on the right shows a hand with detected key points and the GUI elements. The image in the bottom shows the user interface.

Start: The process begins by capturing an image of a hand.

Hand Image: The captured image is processed.

Identifying a palm on hand: MediaPipe's Palm Detector identifies the palm in the image.

Mark 3D hand key points on palm: MediaPipe's Hand Landmark model detects and marks key points on the hand.

Identify hand gesture: The identified key points are mapped to a set of pre-defined gestures, each associated with a specific menu option.

Display menu: Based on the recognized gesture, the corresponding menu is displayed, providing the user with interactive options.

5.4 Data Flow Diagram

The various functions and conditions used in the system are explained in the flow Diagram of the real-time AI virtual mouse System

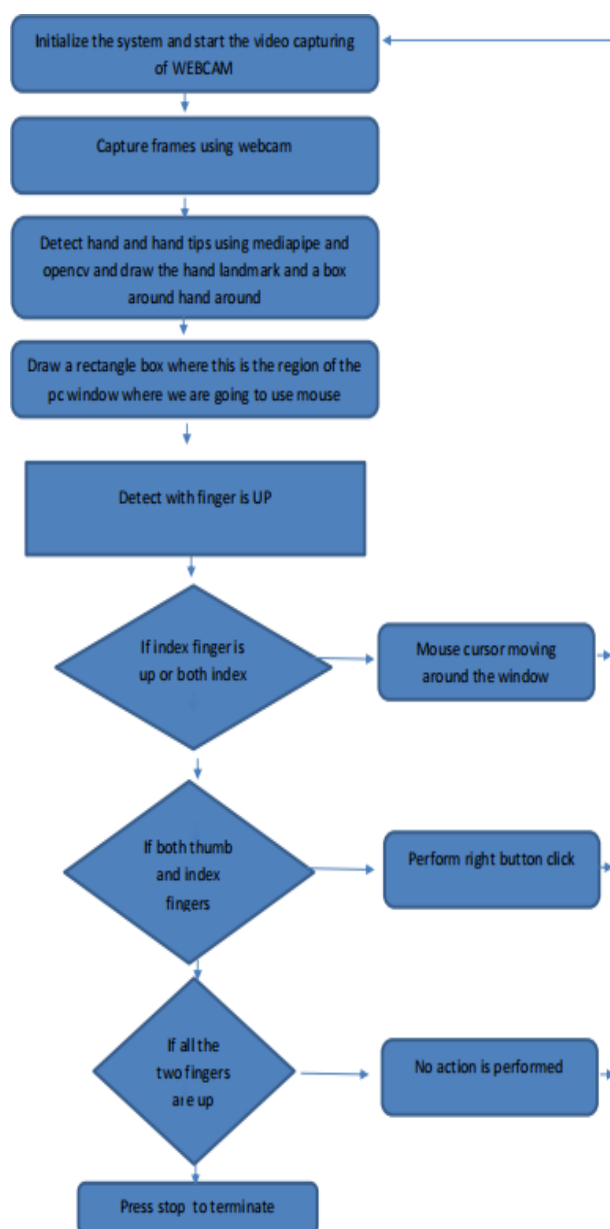


Figure 5.4:Data Flow Diagram

The above Figure 5.4 depicts a Data flow diagram of functions and conditions how the ai virtual mouse System works in real time by using hand gestures.

5.5 Use Case Diagram

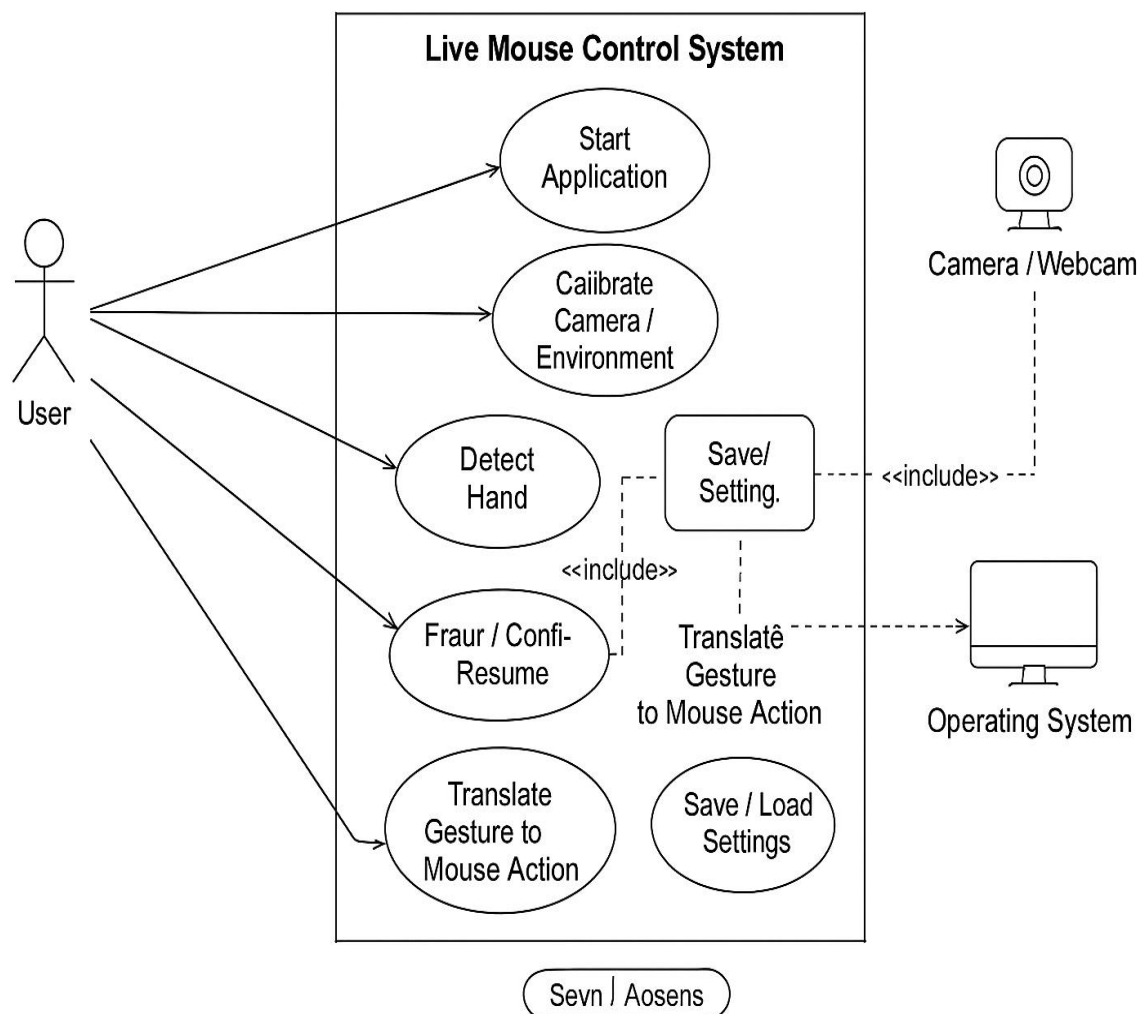


Figure 5.5:Use Case Diagram

The *Live Mouse Control Using Hand Gestures* system lets the user operate the computer mouse using hand movements captured by a webcam. The system detects and tracks the user's hand, converts gestures into actions like moving the cursor, clicking, dragging, and scrolling, and sends these actions to the operating system. The user can start the app, calibrate the camera, adjust settings, pause or resume control, and exit the system, while the application manages errors and ensures smooth gesture recognition.

The above Figure 5.5 shows a Use Case diagram how the live mouse control system works by using os and camera user

CHAPTER 6

IMPLEMENTATION

6.1 Language selection

Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming.

6.1.1 Introduction:

Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.

The Python interpreter is easily extended with new functions and data types implemented in C or C++ (or other languages callable from C). Python is also suitable as an extension language for customizable applications.

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted** – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented** – Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language** – Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

6.1.2 Features:

Python's features include – All these.

- **Easy-to-learn** – Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** – Python code is more clearly defined and visible to the eyes.

- **Easy-to-maintain** – Python's source code is fairly easy-to-maintain.
- **A broad standard library** – Python's bulk of the library is very portable and cross- platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** – Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable** – Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** – you can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases** – Python provides interfaces to all major commercial databases.
- **GUI Programming** – Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable** – Python provides a better structure and support for large programs than shell scripting.

6.1.3 Why is Python preferred as a tools?

It is 'Pythonic' when the code is written in a fluent and natural style. Apart from that, it is also known for other features that have captured the imaginations of data science community.

Easy to learn: The most alluring factor of Python is that anyone aspiring to learn this language can learn it easily and quickly. When compared to other data science languages like R, Python promotes a shorter learning curve and scores over others by promoting an easy-to-understand syntax.

Scalability: When compared to other languages like R, Python has established a lead by emerging as a scalable language, and it is faster than other languages like Matlab and Stata. Python's scalability lies in the flexibility that it gives to solve problems, as in the case of YouTube that migrated to Python. Python has come good for different usages in different industries and for rapid development of applications of all kinds.

6.1.4 Choice of data science libraries:

The significant factor giving the push for Python is the variety of data science/data analytics libraries made available for the aspirants. Pandas, StatsModels, NumPy, SciPy, and Scikit-Learn, are some of the libraries well known in the data science community. Python does not stop with that as libraries have been growing over time. What you thought was a constraint a year ago would be addressed well by Python with a robust solution addressing problems of specific nature.

6.1.5 Python community:

One of the reasons for the phenomenal rise of Python is attributed to its ecosystem. As Python extends its reach to the data science community, more and more volunteers are creating data science libraries. This, in turn, has led the way for creating the most modern tools and processing in Python. The widespread and involved community promotes easy access for aspirants who want to find solutions to their coding problems. Whatever queries you need; it is a click or a Google search away. Enthusiasts can also find access to professionals on Code mentor and Stack Overflow to find the right answers for their queries.

6.1.6 Visual Studio Code

A free, open-source, cross-platform code editor that offers a comprehensive set of features, including IntelliSense, debugging, version control, code navigation, customization, and an integrated terminal, with a vast library of extensions to support a wide range of programming languages and development workflows.

6.1.7 Python 3.12

Python 3.12 is the latest version of the Python programming language, and it consists of several components that work together to execute Python code. Here are the main components of Python 3.12

- 1. Lexer (Tokenizer):** Breaks the source code into a series of tokens, such as keywords, identifiers, literals, and symbols.
- 2. Parser (Syntax Analyzer):** Analyzes the tokens and builds an Abstract Syntax Tree (AST) that represents the source code's syntax.
- 3. Abstract Syntax Tree (AST):** A tree-like data structure that represents the source code's syntax, used for further analysis and execution.
- 4. Semantic Analyzer (Semantic Checker):** Checks the AST for semantic errors, such as type errors, and performs scope analysis.
- 5. Object Model:** Provides a way to represent and manipulate Python objects, including their attributes, methods, and behavior.

6. Built-in Functions and Modules: Provides a set of built-in functions and modules that can be used by Python code, such as `len()`, `print()`, and `math`.

7. Standard Library: A collection of modules and functions that provide additional functionality, such as file I/O, networking, and data structures.

8. Exception Handler: Handles runtime errors and exceptions, providing a way to catch and handle errors in Python code

6.1.8 OPEN CV

OpenCV was started at Intel in 1999 by Gary Bradsky and the first release came out in 2000. Vadim Pisarevsky joined Gary Bradsky to manage Intel's Russian software OpenCV team. In 2005, OpenCV was used on Stanley, the vehicle who won 2005 DARPA Grand Challenge. Later its active development continued under the support of Willow Garage, with Gary Bradsky and Vadim Pisarevsky leading the project. Right now, OpenCV supports a lot of algorithms related to Computer Vision and Machine Learning and it is expanding day-by-day.

Currently OpenCV supports a wide variety of programming languages like C++, Python, Java etc and is available on different platforms including Windows, Linux, OS X, Android, iOS etc. Also, interfaces based on CUDA and OpenCL are also under active development for high-speed GPU operations.

All the OpenCV array structures are converted to-and-from Numpy arrays. So whatever operations you can do in Numpy, you can combine it with OpenCV, which increases number of weapons in your arsenal. Besides that, several other libraries like SciPy, Matplotlib which supports Numpy can be used with this. So OpenCV-Python is an appropriate tool for fast prototyping of computer vision problems. OpenCV (Open Source Computer Vision Library) is an open source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products. Being a BSD-licensed product, OpenCV makes it easy for businesses to utilize and modify the code.

OpenCV (Open Source Computer Vision) is a library of programming functions mainly aimed at real-time computer vision. In simple language it is library used for Image Processing. It is mainly used to do all the operation related to Images.

1. OpenCV Applications

OpenCV is being used for a very wide range of applications which include:

- Street view image stitching
- Automated inspection and surveillance
- Robot and driver-less car navigation and control
- Medical image analysis
- Image/image search and retrieval
- Movies - 3D structure from motion
- Interactive art installations

2. OpenCV Functionality

- Image/image I/O, processing, display (core, imgproc, highgui)
- Object/feature detection (objdetect, features2d, nonfree)
- Geometry-based monocular or stereo computer vision (calib3d, stitching, imagestap)
- Computational photography (photo, image, superres)
- Machine learning & clustering (ml, flann)
- CUDA acceleration (gpu)

6.1.9 MediaPipe

MediaPipe is a framework which is used for applying in a machine learning pipeline, and it is an open-source framework of Google. The MediaPipe framework is useful for cross-platform development since the framework is built using the time series data. The MediaPipe framework is multimodal, where this framework can be applied to various audios and videos. The MediaPipe framework is used by the developer for building and analyzing the systems through graphs, and it also been used for developing the systems for the application purpose.

The steps involved in the system that uses MediaPipe are carried out in the pipeline configuration. The pipeline created can run in various platforms allowing scalability in mobile and desktops. The MediaPipe framework is based on three fundamental parts; they are performance evaluation, framework for retrieving sensor data, and a collection of components which are called calculators, and they are reusable. A pipeline is a graph which consists of components called calculators, where each calculator is connected by streams in which the packets of data flow through.

Benefits:

- **Faster Development:** MediaPipe's modular architecture and pre-built components enable faster development and prototyping of machine learning pipelines.
- **Improved Performance:** MediaPipe's optimized architecture and real-time processing capabilities enable faster and more accurate processing of large datasets.
- **Increased Flexibility:** MediaPipe's extensible architecture and support for multiple data modalities enable developers to adapt to changing requirements and integrate new data sources.

6.1.10 Pyautogui

PyAutoGUI is a Python library used for automating mouse and keyboard actions on a computer. It allows programs to control the system just like a human user by moving the cursor, clicking, typing, scrolling, and taking screenshots. The library works on Windows, macOS, and Linux and does not require complex setup, which makes it ideal for beginners and small automation projects. One of the main features of PyAutoGUI is mouse and keyboard automation. It can move the cursor to specific coordinates, perform left or right clicks, drag objects on the screen, type characters, press special keys, and use key combinations like Ctrl+C or Ctrl+V. It also includes screenshot functionality, allowing scripts to capture the screen and analyze pixel data. Using image recognition, PyAutoGUI can locate images or icons on the screen and interact with them automatically. The library uses operating system-level APIs to send virtual input events that the system interprets as real user actions. It also includes safety features like a fail-safe mechanism: moving the mouse to the top-left corner immediately stops the script. .

6.1.11 Random

The Python random library is a built-in module used to generate pseudo-random numbers and make random selections. It is widely used in applications such as simulations, games, data sampling, cryptography (non-secure), testing, and any program requiring unpredictable behavior. Since it comes pre-installed with Python, it does not require any additional installation.

The library provides several functions to generate random integers, floating-point numbers, and sequences. The random() function returns a random float between 0 and 1, while randint(a, b) generates a random integer in a given range. Functions like uniform(a, b) return random floating values within specified limits.

The random library is simple, lightweight, and powerful for tasks involving randomness. Its wide range of functions makes it ideal for simulations, random testing, password generation (basic), games, and educational projects. Overall, it is an essential module for Python programming that helps introduce the concept of randomness in computing.

6.2 Pseudo Code

Pseudocode is a high-level representation of a programming algorithm or a set of instructions that is written in a natural language style, but with a structured syntax similar to programming languages. It is used to describe the logic and flow of a program without worrying about the specific syntax of a programming language.

```
# import libraries

import cv2
import mediapipe as mp
import pyautogui
import random
import util

from pynput.mouse import Button, Controller

mouse = Controller()

screen_width, screen_height = pyautogui.size()

mpHands = mp.solutions.hands
hands = mpHands.Hands(
    static_image_mode=False,
    model_complexity=1,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7,
    max_num_hands=1
)

def find_finger_tip(processed):
    if processed.multi_hand_landmarks:
        hand_landmarks = processed.multi_hand_landmarks[0] # Assuming only one hand is detected
        index_finger_tip = hand_landmarks.landmark[mpHands.HandLandmark.INDEX_FINGER_TIP]
        return index_finger_tip
    return None, None
```

```
def move_mouse(index_finger_tip):
    if index_finger_tip is not None:
        x = int(index_finger_tip.x * screen_width)
        y = int(index_finger_tip.y / 2 * screen_height)
        pyautogui.moveTo(x, y)
def is_left_click(landmark_list, thumb_index_dist):
    return (
        util.get_angle(landmark_list[5], landmark_list[6], landmark_list[8]) < 50 and
        util.get_angle(landmark_list[9], landmark_list[10], landmark_list[12]) > 90 and
        thumb_index_dist > 50
    )
def is_right_click(landmark_list, thumb_index_dist):
    return (
        util.get_angle(landmark_list[9], landmark_list[10], landmark_list[12]) < 50 and
        util.get_angle(landmark_list[5], landmark_list[6], landmark_list[8]) > 90 and
        thumb_index_dist > 50
    )
def is_double_click(landmark_list, thumb_index_dist):
    return (
        util.get_angle(landmark_list[5], landmark_list[6], landmark_list[8]) < 50 and
        util.get_angle(landmark_list[9], landmark_list[10], landmark_list[12]) < 50 and
        thumb_index_dist > 50
    )
def is_screenshot(landmark_list, thumb_index_dist):
    return (
        util.get_angle(landmark_list[5], landmark_list[6], landmark_list[8]) < 50 and
        util.get_angle(landmark_list[9], landmark_list[10], landmark_list[12]) < 50 and
        thumb_index_dist < 50
    )
def detect_gesture(frame, landmark_list, processed):
    if len(landmark_list) >= 21:
```

```
index_finger_tip = find_finger_tip(processed)

thumb_index_dist = util.get_distance([landmark_list[4], landmark_list[5]])

    if util.get_distance([landmark_list[4], landmark_list[5]]) < 50 and
util.get_angle(landmark_list[5], landmark_list[6], landmark_list[8]) > 90:
    move_mouse(index_finger_tip)
    elif is_left_click(landmark_list, thumb_index_dist):
        mouse.press(Button.left)
        mouse.release(Button.left)
        cv2.putText(frame, "Left Click", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,
255, 0), 2)
    elif is_right_click(landmark_list, thumb_index_dist):
        mouse.press(Button.right)
        mouse.release(Button.right)
        cv2.putText(frame, "Right Click", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,
0, 255), 2)
    elif is_double_click(landmark_list, thumb_index_dist):
        pyautogui.doubleClick()
        cv2.putText(frame, "Double Click", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1,
(255, 255, 0), 2)
    elif is_screenshot(landmark_list, thumb_index_dist):
        im1 = pyautogui.screenshot()
        label = random.randint(1, 1000)
        im1.save(f'my_screenshot_{label}.png')
        cv2.putText(frame, "Screenshot Taken", (50, 50),
cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 0), 2)
def main():
    draw = mp.solutions.drawing_utils
    cap = cv2.VideoCapture(0)
```

```
try:
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        frame = cv2.flip(frame, 1)
        frameRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        processed = hands.process(frameRGB)
        landmark_list = []
        if processed.multi_hand_landma:
rks
        hand_landmarks = processed.multi_hand_landmarks[0] # Assuming only one hand is
detected
            draw.draw_landmarks(frame, hand_landmarks,
mpHands.HAND_CONNECTIONS)
            for lm in hand_landmarks.landmark:
                landmark_list.append((lm.x, lm.y))
            detect_gesture(frame, landmark_list, processed)
            cv2.imshow('Frame', frame)
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break
        finally:
            cap.release()
            cv2.destroyAllWindows()
if __name__ == '__main__':
    main()
```

CHAPTER 7

TESTING

A test case is a series of conditions where a tester decides whether the program meets the criteria and operates correctly. Single functional test is a test case. Test case is a series of step-by-step instructions for checking that something acts as it is appropriate to function. The test cases are written by a QA Team individual. It will not include unit checks, performed by the development team, but in this article we do not get into unit tests. Ensure that anyone writing test cases has decent writing skills and understands the purpose and value of test cases.

7.1 Types of Tests

- Unit testing
- Integration testing
- Functional test
- Black Box Testing

7.2 Unit Testing

Unit testing involves developing test cases validating the internal logic of the programme working properly, and inputting the software generates valid outputs.

1. **Camera Input:** The process starts with capturing a live video feed from the camera.
2. **Finished Image:** The incoming video frame is processed and a "finished image" is generated, likely with some pre-processing steps like noise reduction or frame rate control
3. **Hand Detection (with Special Conditions):** The "finished image" is then analyzed for hand presence using a Hand Detection algorithm.

Importantly, this step is skipped under two conditions:

- On the very first frame of the video.
 - If no hand is detected in the previous frame.
4. **Detection to Rectangle:** If a hand is detected, the bounding box (rectangle) around the hand is determined. This rectangle helps in focusing on the hand area for further processing.
 5. **Cropping:** The image is cropped to the rectangle containing the detected hand, creating a "cropped image" that only focuses on the hand.
 6. **Landmark Detection (with Hand Flag):** The cropped hand image is analyzed for specific landmark points (like fingertips, wrist) using a HandLandmark algorithm. A "Hand Flag" is used

to indicate whether landmarks were successfully detected on the hand. If not, it suggests that the hand might be obscured or out of view.

7. **Landmark to Rectangle:** If landmarks are detected, their positions are refined, likely by fitting them to a more precise rectangular bounding box
8. **Final Output:** The final processed image with annotations (like the bounding rectangle and landmarks) is displayed as "Rendered Image".
9. **Temporal Back Edge:** This blue edge represents a connection to the next frame of the video, suggesting that the entire process will repeat for subsequent frames in the real-time video feed. The flow shows a sequential processing approach, starting with hand detection and then moving to landmark detection.

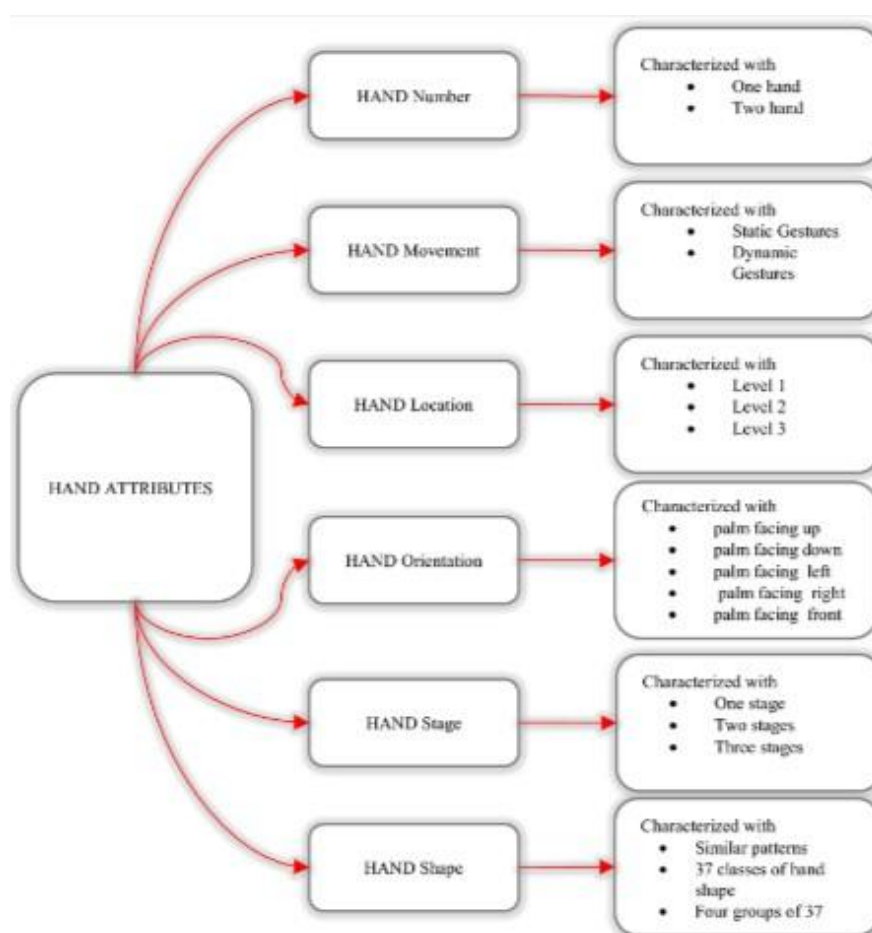


Figure 7.1: Unit Testing

This above figure 7.1, flowchart provides a simplified overview of the pipeline, and additional steps might be involved in a real-world implementation.

The "Reject Hand Flag Landmarks" block emphasizes the importance of filtering out invalid hand detections before landmarking. The blue edge represents a temporal connection, suggesting that information from the previous frame may be used in the current frame's processing.

7.3 Integration Testing

Integration checks are designed to test components of the integrated program to determine whether they are Typically run as a single programmer.

Test Case 1: Hand Detection

Test that the system can detect a hand in the camera's field of view.

Verify that the system can accurately identify the location of the hand and its fingers.

Test that the system can detect multiple hands in the camera's field of view.

Test Case 2: Gesture Recognition

Test that the system can recognize a specific gesture (e.g. a thumbs up or a wave) and respond accordingly.

Verify that the system can accurately distinguish between different gestures.

Test that the system can recognize gestures in different lighting conditions and orientations.

Test Case 3: Application Switching

Test that the system can switch between open applications in response to a recognized gesture.

Verify that the system can accurately switch to the correct application based on the gesture performed.

Test that the system can switch applications correctly in response to multiple gestures (e.g., a series of swipe-right or swipe-left gestures).

Test Case 4: Gesture Thresholding

Test that the system can accurately detect the threshold for a gesture (e.g. the minimum distance between the thumb and index finger for a thumbs up gesture).

Verify that the system can adjust the threshold based on user input or other factors.

Test that the system can accurately detect gestures with varying levels of precision.

Test Case 5: Interference and Noise

Test that the system can accurately detect gestures in the presence of interference or noise (e.g. background movement or lighting changes).

Verify that the system can filter out irrelevant data and focus on the gesture.

Test that the system can accurately detect gestures in different environments (e.g. indoors vs. outdoors).

Test Case 6: User Calibration

Test that the system can accurately calibrate to a user's specific hand shape and gesture style.

Verify that the system can adjust its detection and recognition algorithms based on user input.

Test that the system can accurately detect gestures after calibration.

Test Case 7: Error Handling

Test that the system can handle errors and exceptions gracefully (e.g. if the camera is not available or the gesture is not recognized).

Verify that the system can provide feedback to the user in case of an error.

Test that the system can recover from errors and continue to function correctly.

7.4 System Testing

System testing for the gesture-controlled mouse project evaluates the entire workflow end-to-end using the actual webcam, MediaPipe hand-tracking, and OS mouse-control libraries. The goal is to verify whether cursor movement, left/right click, double click, and screenshot actions trigger correctly across different users, lighting conditions, and gesture variations. Tests include normal operation, edge-of-frame gestures, rapid gesture sequences, background interference, and incorrect or partial hand detection. The system must respond smoothly, avoid false positives, and remain stable without crashing, even when multiple hands appear or the webcam feed becomes unreliable. This ensures the program behaves reliably in real-world usage and handles all operational scenarios gracefully.

7.5 Acceptance Testing

Acceptance testing confirms that the gesture mouse-control system meets all functional expectations and is ready for practical use. The system is accepted if it consistently recognizes intended gestures for cursor movement, clicking, double-clicking, and screenshot capture with high accuracy, while keeping false triggers minimal. It must also handle consecutive gestures without lag, provide on-screen feedback, and operate correctly under normal indoor lighting for different users. Additional acceptance checks verify that screenshots save properly, the program exits cleanly, and incorrect or ambiguous gestures do not cause unexpected actions. If the system performs reliably across these conditions, it is considered ready for deployment..

7.6 Test Cases

Test cases for the gesture-based mouse system define clear, repeatable scenarios that evaluate gesture detection accuracy and system response. Each case specifies the gesture input, conditions (lighting, position, hand pose), execution steps, and the expected output such as cursor movement, left/right click, double click, or screenshot generation. The set includes normal gesture recognition, edge-of-frame performance, rapid gesture sequences, incorrect gesture rejection, partial hand detection handling, multi-hand interference, camera failure handling, and verification of screenshot file creation. These structured test cases ensure every feature, boundary condition, and error scenario is validated fully and consistently.

Test Case 1

Table 7.1: Hand detection and cursor control

Test case	TC-1
Test case description	Hand Detection and Cursor control
Input	A video feed with a hand performing various gestures (e.g., up/down, left/right, circular).
Expected output	Cursor can access and click any folder.
Actual output	Record the actual changes based on the performed gestures.
Pass/Fail	Pass

The Above Table 7.1 shows the test case and TC-1 represented as

Objective: Verify that the system accurately recognizes user hand gestures and can access any cursor movements.

Test Case: Perform various hand gestures (e.g., hand up, hand down, hand moving left, hand moving right) in front of the camera..

Expected Result: The system should respond to the gestures with high accuracy (at least 95%).

Test Case 2

Table 7.2: Gesture Sensitivity and Accuracy

Test case

TC-2

Test case description	Gesture Sensitivity and Accuracy
Input	A video feed with the hand performing a variety of gestures, including subtle and fast movements.
Expected output	The system should accurately detect and respond to subtle gestures as well as faster movements.
Actual output	Record the system's response to each gesture
Pass/Fail	Pass

The Above Table 7.2 shows the test case and TC-2 represented as

Objective: Verify the accuracy and sensitivity of the system's gesture recognition feature.

Test Case: The system should be able to accurately recognize the various gestures performed by the user.

Expected Result: The system should recognize all gestures with a high level of accuracy and sensitivity (at least 95%).

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

This project demonstrates a system that uses hand gestures as an effective and intuitive method for controlling mouse operations through computer vision. The gesture-based mouse controller does not require any external devices or markers, making it practical for real-life use on basic computers equipped with standard webcams. The system successfully meets its functional requirements by accurately recognizing gestures for cursor movement, left and right clicks, double clicks, and screenshot capture, and performing these actions reliably on the computer. It also fulfills the non-functional requirements, as it operates with good speed, responsiveness, and ease of use, providing a smooth interaction experience. Safety considerations are addressed by ensuring the system performs actions only when intentional gestures are made, preventing accidental clicks or system misuse. These achievements are largely made possible by the powerful combination of MediaPipe and OpenCV, which provide robust hand-tracking and landmark-detection capabilities. By continuously interpreting fingertip positions and gesture patterns, the system automates essential computer interactions and simplifies user control. Overall, this project successfully proves that gesture-based control can offer a hands-free, accessible, and modern alternative to traditional input devices.

8.2 Future Enhancement

The hand gesture recognition system has a wide range of applications in human–computer interaction, touchless navigation, accessibility assistance, gaming, virtual workspaces, and many other modern computing fields. The system we developed for gesture-based mouse control using MediaPipe and OpenCV has shown promising results and demonstrates strong potential for further improvement and expansion in the future. By allowing users to control cursor movement, clicks, and screenshots through simple hand gestures, the project highlights how natural interactions can replace traditional input devices. The hand gesture recognition system has a wide range of applications in human–computer interaction, touchless navigation, accessibility assistance, gaming, virtual workspaces, and many other modern computing fields. The system we developed for gesture-based mouse control using MediaPipe and OpenCV has shown promising results and demonstrates strong potential for further improvement and expansion in the future. By allowing users to control cursor movement, clicks, and screenshots through simple hand gestures, the project highlights how natural interactions can replace traditional input devices.

BIBLIOGRAPHY

- [1]. Joy Guha, Shreya Kumari, Prof. Shiv Kumar Verma, “AI Virtual Mouse Using Hand Gesture Recognition”, Applied Science & Engineering Technology, Volume IJRASET41981, IJRASET, April 2022.
- [2]. S.Shriram, “Deep Learning-Based Real-Time AI Virtual Mouse System Using Computer Vision”, Applied Sciences, Volume 2021, Article ID 8133076, 31 Jul 2021, Hindawi
- [3]. Abhik Banerjee, Abhirup Ghosh, Koustuvmoni Bharadwaj, Hemanta Saikia “Mouse Control using a Web Camera based on Colour Detection” IJCTT, March 2014, volume 9 number 1, ISSN:2231-2803
- [4]. Hyung-Jeong Yang, et, al. Real-time virtual mouse system using RGB-D images and f finger tip detection in Multimedia Tools and Applications, 2020,<https://doi.org/10.1007/s11042-020-10156-5>
- [5]. Nisha Patil, Niketan Bothe, Shivam Gulve, “Image Scene Understanding -Object Detection in Aerial Images using Convolutional Neural Networks”, International Journal for Scientific Research & Development, Vol. 8, Issue 10, 2020.
- [6]. Pratik Joshi, S. Kulkarni, “Virtual Mouse Using Hand Gesture Recognition”, International Journal of Computer Applications (IJCA), 2021.
- [7]. Y. Lin et al., “Real-Time Hand Gesture Monitoring Model Based on MediaPipe’s Registerable System,” Sensors, vol. 24, no. 19, 2024.

APPENDIX A – ACRONYMS

- AR: Augmented Reality
- VR: Virtual Reality
- NLP: Natural Language Processing
- OpenCV: Open Source Computer Vision Library
- Colour: Red
- API: Application Programming Interface
- CNN: Convolutional Neural Network
- SRS: Software Requirements Specification

APPENDIX B

SNAPSHOTS

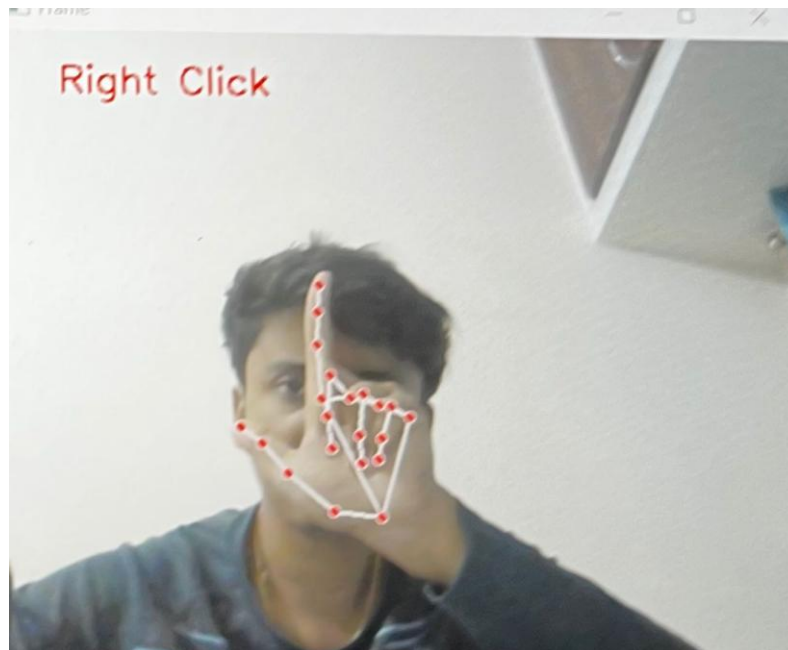


Figure B1: Right click using Hand Gesture

The above Figure B1 shows a hand with a few fingers extended with red colors passing through the palm-co ordinates.

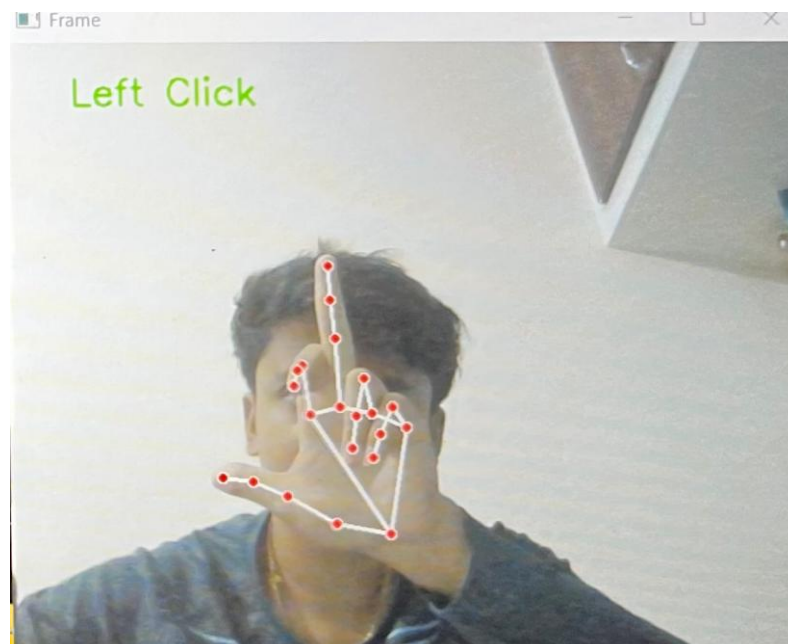


Figure B2: Left click using Hand Gesture

The above Figure B2 shows the left click using middle finger.

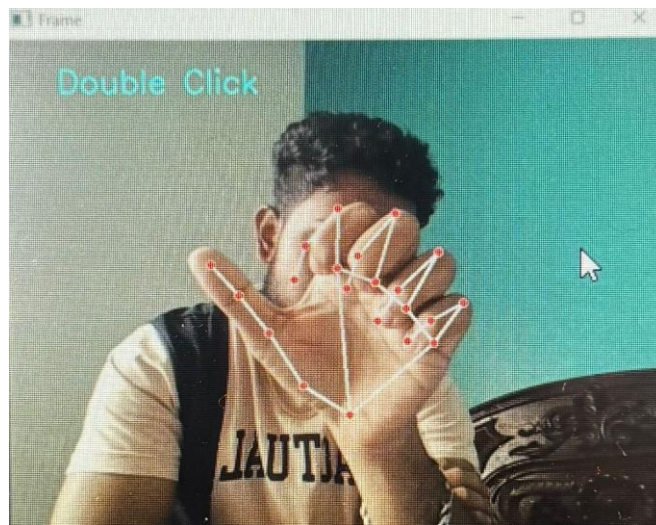


Figure B3: Double click using hand gesture.

The above Figure B3 shows the Gesture of hand of double click.

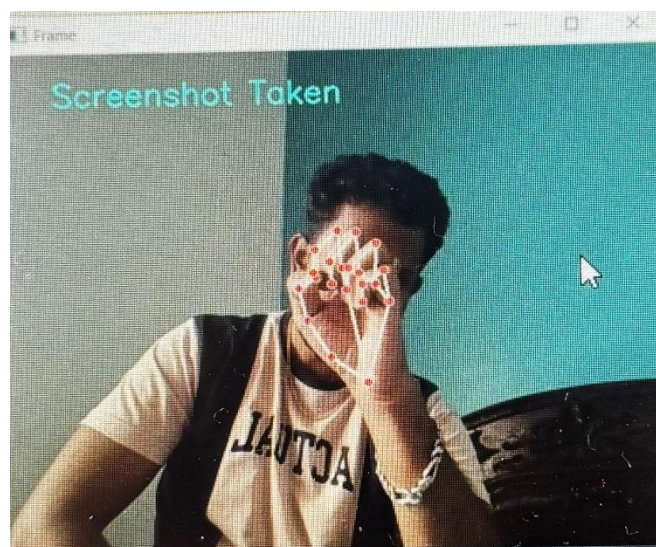


Figure B4: Screenshot using hand gesture

The above Figure B4 shows taking a screenshot using the gesture of hand.